

표적 궤적 모의

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그리기까지

개념편 · 코드 구현은 다음 발표에서

레이다 시스템 소프트웨어 스터디 · Part 3

발표 대본

표지 · 약 20초

안녕하십니까. 이번 과제의 주제는 표적 궤적 모의입니다. 지난번에는 레이다가 켜 값을 위경도로 바꾸는 일을 했는데, 이번에는 반대로 표적을 우리가 직접 만들어서 60초 동안 움직여 보는 일입니다.

오늘은 코드보다 원리 위주로 말씀드리겠습니다. 구현과 실행 결과는 다음 시간에 이어서 하겠습니다.

INDEX

목차

- | | | |
|-----------|---------------|--|
| 00 | 들어가며 | 과제 3이 묻는 것
과제 2와 무엇이 다른가 |
| 01 | 표적을 움직이는 원리 | 좌표계 넷 · 위치와 속도
시간 적분 · 지구 곡률 |
| 02 | 기동 모델 | 하중배수 g · 원운동
세 축 · 기동 시간표 |
| 03 | 구조체와 시뮬레이션 설계 | 표적 카드 · 전체 흐름 · 조건
예상 결과 · 검증 · 다음 단계 |

오늘은 개념편입니다. 수학은 고등학교 물리의 등속 원운동 정도면 충분하고, 나올 때마다 다시 설명합니다.

2

발표 대본

목차 · 약 40초

네 부분으로 나뉘었습니다.

먼저 들어가며에서 과제가 무엇을 요구하는지, 그리고 지난 과제와 뭐가 달라졌는지를 짚습니다.

첫째 장은 표적을 움직이는 원리입니다. 어느 좌표계를 쓰는지, 위치와 속도를 왜 다르게 다뤄야 하는지, 시간을 어떻게 흘려보내는지, 그리고 지구가 둥글다는 게 왜 문제가 되는지를 봅니다.

둘째 장은 기동 모델입니다. 표적이 방향을 바꾸는 걸 몇 g 라는 숫자 하나로 어떻게 표현하는지 다룹니다.

셋째 장은 설계입니다. 표적 구조체를 어떻게 잡을지, 프로그램 전체가 어떻게 돌아갈지, 그리고 결과를 어떻게 확인할지 정리하겠습니다.

수학은 고등학교 물리에 나오는 등속 원운동 정도면 충분합니다. 나올 때마다 다시 설명드리겠습니다.

00. 과제 3이 묻는 것

표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라



시뮬레이션 시간

60 초 · 0.1 초 간격

- $t = 0.0 \text{ s}$
- $t = 0.1 \text{ s}$
- $t = 0.2 \text{ s}$
- $t = 0.3 \text{ s}$
- $\vdots$
- $t = 60.0 \text{ s}$

위치 갱신 600 번

표적마다 · 위경도로 기록

① 표적 구조체를 정의한다

표적 최대 10개 · 표적마다 기동 최대 30개
초기 위치는 위경도·고도, 속도는 동체 속력 하나로

② 위치·속도는 ECEF 에서 갱신한다

동체 속력을 ECEF 성분으로 바꾸는 방법도 고민 (추가 항목)

③ 60초 동안 0.1초 간격으로

플랫폼 하나 (정지) · 대함 표적 · 대공 표적

④ 위경도로 출력하고 그림으로

플랫폼과 표적의 궤적을 지도 위에

한 줄로 : 표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라

표적마다 600개의 점이 찍힌다. 지난 과제의 좌표변환 함수는 그대로 쓰고, 그 위에 시간과 기동을 얹는다.

발표 대본

들어가며 · 약 70초

과제를 한 줄로 줄이면 이렇습니다. 표적을 정의하고, 0.1초마다 움직이고, 위경도로 그려라.

왼쪽 그림이 상황입니다. 우리 배는 북위 32도, 동경 126도에 서 있고 움직이지 않습니다. 북쪽으로 13.3 킬로미터 떨어진 곳에 대공 표적이 고도 300 미터에서 초속 200 미터로 남쪽, 그러니까 우리 쪽으로 날아옵니다. 그 옆에 대함 표적이 초속 30 미터로 서쪽으로 갑니다. 이걸 60초 동안, 0.1초 간격으로 위치를 계속 갱신하는 겁니다. 그러면 표적마다 600개의 점이 찍힙니다.

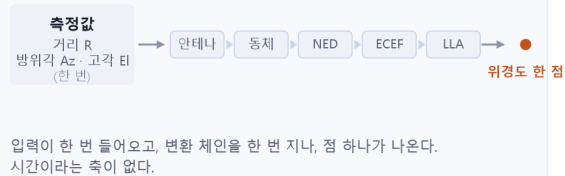
오른쪽이 과제가 요구하는 네 가지입니다. 하나, 표적 구조체를 정의할 것. 표적은 최대 10개, 표적마다 기동은 최대 30번까지 답을 수 있어야 합니다. 둘, 표적의 위치와 속도는 ECEF 좌표계에서 갱신할 것. 셋, 시뮬레이션 조건은 60초, 0.1초 간격. 넷, 결과는 위경도로 출력하고 그림으로 보여 줄 것.

여기에 추가 고민 항목이 하나 있는데, 표적의 동체 속도를 ECEF 성분으로 바꾸는 방법입니다. 이걸 뒤에서 한 장을 따로 잡고 설명드리겠습니다.

00. 과제 2와 무엇이 다른가

한 점을 한 번 변환하던 일이, 매 0.1초 600번 반복하는 일이 된다

과제 2 · 한 점을 한 번 변환



같은 것 — 좌표변환 함수 넷은 그대로

LLA ↔ ECEF, 동체 → NED, NED → ECEF. 제공된 참고 코드에 그대로 있다. 안테나 좌표계만 이번엔 등장하지 않는다 (재는 쪽이 아니라 만드는 쪽).

과제 3 · 매 0.1 초마다, 60 초 동안 반복



새로운 것 — 속도 · 시간 · 기동

속도 : 화살표는 위치와 다르게 변환한다 (회전만)
시간 : 새 위치 = 지금 위치 + 속도 × 0.1 s. 기동 : 몇 g 로 어느 축을 도는가

과제 2의 변환 체인을 재료를 쓰고, 그 위에 시간 루프와 기동 모델을 얹는다

4

발표 대본

들어가며 · 약 60초

지난 과제와 뭐가 다른지부터 분명히 하고 가겠습니다.

왼쪽이 과제 2입니다. 측정값 하나가 들어오면 변환 체인을 한 번 지나서 위경도 한 점이 나옵니다. 시간이라는 게 없었습니다.

오른쪽이 이번 과제입니다. 초기값을 한 번 넣고 나면, 그다음부터는 같은 계산을 600번 반복합니다. 기동을 보고, 자세를 고치고, 속도를 구하고, 위치를 옮기고. 한 바퀴 돌 때마다 점이 하나씩 찍혀서 나중에 선이 됩니다. 새로 들어온 건 시간 하나입니다.

그러면 지난번에 만든 건 어떻게 되느냐. 그대로 씁니다. 위경도와 ECEF를 오가는 함수, 동체에서 NED로 가는 함수, NED에서 ECEF로 가는 함수. 이 넷은 이번에 제공된 참고 코드에도 그대로 들어 있고, 손댈 일이 없습니다. 안테나 좌표계만 이번엔 안 나옵니다. 레이다가 재는 게 아니라 표적을 만드는 쪽이니까요.

새로 배워야 할 건 셋입니다. 속도, 시간, 기동. 지금부터 이 셋을 차례로 보겠습니다.

01

표적을
움직이는 원리

- 1 이번에 쓰는 좌표계 넷 — LLA · ECEF · NED · 동체
- 2 위치와 속도는 다르게 변환한다
- 3 동체 속도를 ECEF 속도로 — 회전 두 번
- 4 시간 적분 — 새 위치 = 지금 위치 + 속도 × 0.1 초
- 5 지구는 둥글다 — 60초 직진하면 11 m 떠오른다
- 6 한 스텝의 순서 — 다섯 단계

이 발표의 약속 : 각도는 도(°)로 말하고 프로그램 안에서는 라디안. 거리는 m, 시간은 s.

5

발표 대본

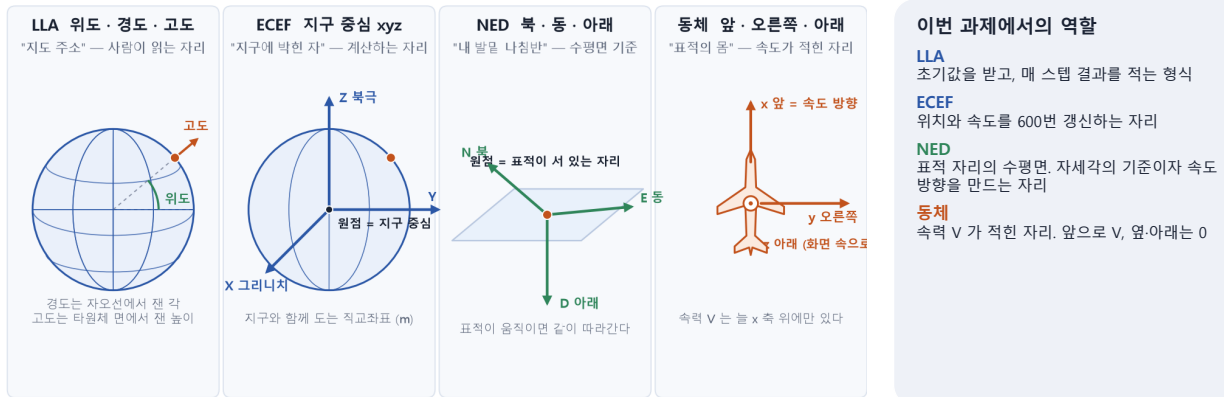
장절 · 약 25초

첫째 장입니다. 표적 하나를 0.1초 뒤로 옮기려면 뭐가 필요한지 원리부터 보겠습니다. 좌표계 넷을 다시 보고, 위치와 속도가 왜 다른지, 시간을 어떻게 더하는지, 그리고 지구가 둥글어서 생기는 문제까지 다섯 장입니다.

미리 약속 하나만 드리면, 발표에서는 각도를 도로 말하고 프로그램 안에서는 라디안을 씁니다. 거리는 전부 미터입니다.

01. 이번에 쓰는 좌표계 넷

입력, 출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체



원점이 표적 자신인 NED가 하나 더 필요하다 — 자세각과 수평면은 표적이 서 있는 자리 기준이기 때문
지난 과제의 NED는 플랫폼 자리였다. 이번 NED는 표적이 움직이면 같이 따라간다.

6

발표 대본

원리 · 약 80초

지난번에 여섯 개를 봤는데 이번에 쓰는 건 넷입니다. 각각 역할이 다릅니다.

첫째, 위도 경도 고도. 지도 주소라고 생각하시면 됩니다. 사람이 읽는 자리라서, 과제도 표적의 처음 위치를 이 형식으로 받고 결과도 이 형식으로 내라고 합니다.

둘째, ECEF. 지구 중심에 원점을 두고 미터 단위로 x, y, z를 잴 직교좌표입니다. 지구에 박아 놓은 자라고 보시면 됩니다. 계산은 전부 여기서 합니다. 왜 여기서 하는지는 다음 장에서 말씀드리겠습니다.

셋째, NED. 표적이 서 있는 자리에서 북쪽, 동쪽, 아래쪽으로 잴 좌표입니다. 내 발밑 나침반입니다. 중요한 건 원점이 플랫폼이 아니라 표적 자신이라는 점입니다. 표적이 움직이면 이 나침반도 같이 따라갑니다. 이걸 왜 굳이 두느냐 하면, 수평면이 필요하기 때문입니다. 자세각이라는 게 수평면 기준으로 정의되니까요.

넷째, 동체. 표적의 몸 기준입니다. 앞, 오른쪽, 아래. 과제가 표적 속도를 동체 속도, 그러니까 앞으로 가는 속력 하나로 받으라고 했는데, 그 속력이 적혀 있는 자리가 여기입니다. 앞으로 초속 200미터, 이게 전부입니다. 옆이나 아래 성분은 없습니다.

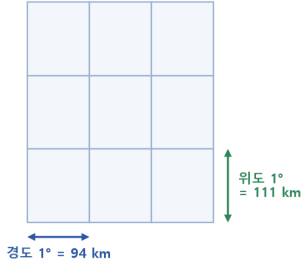
정리하면 입력과 출력은 위경도, 계산은 ECEF, 수평면은 NED, 속력은 동체입니다.

01. 왜 ECEF 에서 움직이나

위경도는 각도라 더할 수 없고, ECEF 는 미터라 더하면 끝난다

위경도 격자 (북위 32° 부근)

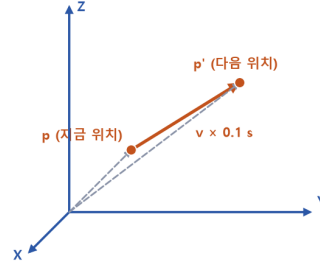
"1도" 라는 같은 눈금인데 실제 길이가 다르다



✗ 위도 + 속도 × 시간? 각도에 미터를 더할 수 없다

ECEF 직교좌표 (미터)

세 축이 모두 미터라 벡터를 그대로 더한다



✓ $p' = p + v \times \Delta t$ 더하기 한 줄이면 끝

① 위경도는 각도다

위도 1° 는 111 km, 경도 1° 는 94 km (32°N).
각도에 미터를 더할 수 없다.

② ECEF 는 미터다

세 축이 전부 미터라 벡터를 그대로 더한다.
 $p' = p + v \times \Delta t$, 한 줄.

③ 플랫폼과 무관한 절대 좌표

플랫폼이 움직이거나 흔들려도 표적 궤적은 그대로.
플랫폼 기준 NED 에서 움직이면 플랫폼 흔들림이 표적에 섞인다.

흐름 : 위경도로 받는다 → ECEF 로 바꾼다 → 600번 내내 ECEF 에서 더한다 → 기록할 때만 위경도로 되돌린다

과제 요구 (2) "표적 상태는 ECEF 좌표계에서 갱신" 의 이유가 이것이다.

7

발표 대본

원리 · 약 70초

과제가 굳이 ECEF에서 갱신하라고 못 박은 이유를 보겠습니다.

왼쪽이 위경도 격자입니다. 북위 32도 근처에서 위도 1도는 111 킬로미터인데, 경도 1도는 94 킬로미터밖에 안 됩니다. 같은 1도라는 눈금인데 실제 길이가 다릅니다. 게다가 위도는 각도입니다. 위도에다 초속 200 미터 곱하기 0.1초, 그러니까 20 미터를 더한다는 건 말이 안 됩니다. 도에 미터를 더할 수 없으니까요.

오른쪽이 ECEF입니다. 세 축이 전부 미터입니다. 지금 위치 p 에 속도 곱하기 시간을 더하면 다음 위치 p' 이 됩니다. 벡터 덧셈 한 줄이면 끝입니다.

이유가 하나 더 있습니다. ECEF는 플랫폼과 아무 상관이 없는 절대 좌표입니다. 이번엔 플랫폼이 서 있지만, 나중에 플랫폼이 움직이더라도 표적 궤적은 흔들리지 않습니다. 플랫폼 기준 NED에서 표적을 움직이면 플랫폼이 흔들릴 때마다 표적도 같이 흔들리게 됩니다.

그래서 흐름은 이렇습니다. 위경도로 받아서 ECEF로 바꾸고, 600번 내내 ECEF에서 더하고, 기록할 때만 위경도로 되돌립니다.

01. 위치와 속도는 다르게 변환한다

위치는 원점이 있어야 말이 되고, 속도는 화살표 하나면 된다

기동 판단

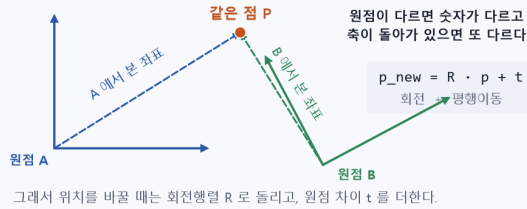
자세 갱신

속도 변환

위치 적분

LLA 변환

위치 — "어디에 있나" 는 원점이 있어야 말이 된다



위치 : $p_{\text{new}} = R \cdot p + t$

회전하고 원점 차이만큼 옮긴다. 지난 과제에서 계속 하던 그 일.

속도 — "어느 쪽으로 얼마나 빨리" 는 화살표 하나면 된다



속도 : $v_{\text{new}} = R \cdot v$

축이 돌아간 만큼만 돌린다. t를 더하면 그 순간 속도가 아니게 된다.

과제의 "추가 고민" — 동체 속도 V_{heading} 을 ECEF 성분으로 바꾸는 방법 — 의 답이 오른쪽 한 줄이다

8

발표 대본

원리 · 약 80초

여기가 이번 과제의 핵심 하나입니다. 과제에 추가 고민 항목으로 나온, 동체 속도를 ECEF 성분으로 바꾸는 방법의 답이기도 합니다.

왼쪽, 위치입니다. 어디에 있나라는 말은 원점이 있어야 성립합니다. 같은 점 P를 원점 A에서 보면 어떤 숫자고, 원점 B에서 보면 다른 숫자입니다. 축이 돌아가 있으면 또 다릅니다. 그래서 위치를 다른 좌표계로 옮길 때는 회전행렬로 돌리고 원점 차이를 더합니다. 지난 과제에서 계속 했던 그겁니다.

오른쪽, 속도입니다. 어느 쪽으로 얼마나 빨리라는 말에는 원점이 필요 없습니다. 화살표 하나면 됩니다. 그림에서 같은 화살표를 네 군데 그려 봤는데, 어디에 그려도 같은 화살표입니다. 원점을 옮겨도 화살표는 그대로입니다. 그러니까 속도를 다른 좌표계로 옮길 때는 축이 돌아간 만큼만 돌려 주면 됩니다. 회전만 하고 평행이동은 하지 않습니다.

이걸 헛갈리면 어떻게 되느냐. 속도 벡터에 원점 차이를 더해 버리면 그 순간 그건 속도가 아니게 됩니다. 크기가 수천 킬로미터짜리 화살표가 됩니다. 뒤에서 실제 숫자로 보여 드리겠습니다.

식으로는 위치가 R 곱하기 p 더하기 t, 속도가 R 곱하기 v입니다. t가 있고 없고 차이입니다.

01. 동체 속도를 ECEF 속도로 — 회전 두 번

자세각으로 한 번, 표적 자신의 위도·경도로 한 번. 평행이동은 없다



9

발표 대본

원리 · 약 85초

앞 장의 원리를 실제 순서로 옮기면 이 그림입니다. 회전 두 번입니다.

첫째 칸, 동체 좌표의 속도. 표적은 앞으로만 갑니다. 그래서 속도 벡터가 V, 0, 0 입니다. 대공 표적이면 200, 0, 0. 속력 하나만 받아도 벡터가 정해지는 이유가 이겁니다.

둘째 칸, NED 좌표의 속도. 자세각 롤, 피치, 요로 회전합니다. 지난 과제의 동체에서 NED로 가는 행렬을 그대로 씁니다. 대공 표적은 요가 180도, 남쪽을 보고 있으니까 결과가 마이너스 200, 0, 0 입니다. 북쪽 성분이 마이너스 200, 즉 남쪽으로 200 이라는 뜻입니다.

셋째 칸, ECEF 좌표의 속도. 위도와 경도로 한 번 더 회전합니다. 지난 과제의 NED에서 ECEF로 가는 행렬입니다. 다만 두 가지를 조심해야 합니다. 하나, 이 위도 경도는 플랫폼 것이 아니라 표적 자신의 것입니다. 표적이 서 있는 자리의 수평면이니깐요. 둘, 참고 코드의 이 함수는 마지막에 플랫폼 위치를 더하게 되어 있습니다. 위치를 바꿀 때는 맞는데 속도에는 그 항이 있으면 안 됩니다. 그래서 플랫폼 위치 인자에 0을 넣어야 합니다.

결과가 마이너스 62.5, 86.0, 마이너스 169.4 이고 크기는 정확히 200.0 입니다. 검산 방법이 이겁니다. 회전만 했으니 크기가 변하면 안 됩니다. 실수로 플랫폼 위치를 더하면 크기가 초속 6,372 킬로미터가 나옵니다. 지구 반지름 값이 속도에 섞여 들어간 겁니다. 한 번 보면 바로 잡히는 실수입니다.

01. 시간 적분 — 새 위치 = 지금 위치 + 속도 × 0.1 초

한 스텝 안에서는 속도가 일정하다고 본다. 그래서 곱셈 한 번이면 정확하다

기동 판단

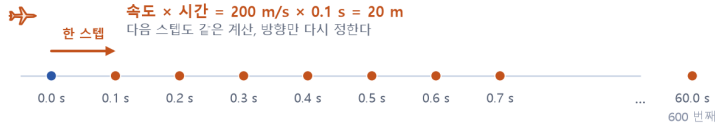
자세 갱신

속도 변환

위치 적분

LLA 변환

자동차로 치면 : 시속 720 km 로 0.1 초 가면 20 m. 그걸 600 번 더하면 12 km.
속도의 방향은 매 스텝 다시 계산하므로 (앞 장의 회전 두 번), 곡선도 이 방식으로 그려진다.



$$\text{새 위치} = \text{지금 위치} + \text{속도} \times 0.1 \text{ s}$$

$$p(t + \Delta t) = p(t) + v(t) \cdot \Delta t \quad \text{한 스텝 안에서는 속도가 일정하므로 이 한 줄로 충분하다}$$

왜 이렇게 단순해도 되나

0.1초 동안은 방향도 크기도 안 바뀐다고 본다.
곡선은 매 스텝 방향을 다시 정해서 만든다.

문지쿠타 4차는?

가속도가 위치의 함수인 궤도 문제용.
이번엔 속도를 우리가 정해 주므로 필요 없다.

숫자 감

대공 한 스텝 20 m · 60초 12 km, 대함 한 스텝 3 m · 60초 1.8 km

$$p(t + \Delta t) = p(t) + v(t) \cdot \Delta t \quad \text{ECEF의 } x, y, z \text{ 세 성분에 각각}$$

$\Delta t = 0.1 \text{ s}$, 600 스텝. $v(t)$ 는 직전 단계(속도 변환) 에서 매 스텝 새로 만든 값이다.

10

발표 대본

원리 · 약 70초

속도를 구했으니 이제 시간을 흘려보냅니다. 방법은 초등학교 산수입니다. 새 위치는 지금 위치 더하기 속도 곱하기 0.1초.

자동차로 치면 이렇습니다. 시속 720 킬로미터는 초속 200 미터인데, 0.1초 동안 가면 20 미터입니다. 그걸 600번 더하면 12 킬로미터. 대공 표적이 60초 동안 가는 거리가 그겁니다. 대함 표적은 초속 30 미터니까 한 스텝에 3 미터, 60초에 1.8 킬로미터입니다.

이렇게 단순해도 되는 이유는 한 스텝 안에서 속도가 일정하기 때문입니다. 0.1초 동안은 방향도 크기도 안 바뀐다고 보는 겁니다. 곡선은 어떻게 그리느냐 하면, 매 스텝 방향을 다시 정해서 만듭니다. 조금씩 방향이 바뀌는 직선 600개를 이으면 곡선이 됩니다.

지난번 참고 자료에 나왔던 문지쿠타 4차 같은 건 안 씁니다. 그건 위성 궤도처럼 가속도가 위치에 따라 변하는 문제에 필요한 거고, 이번엔 속도를 우리가 직접 정해 주니까 이 한 줄이면 정확합니다.

ECEF 세 성분 각각에 이 한 줄을 적용하면 됩니다.

01. 지구는 둥글다 — 60초 직진하면 11 m 떠오른다

처음 속도를 그대로 쓰면 접선을 따라 지구에서 멀어진다. 매 스텝 수평면을 다시 잡아야 한다



11

발표 대본

원리 · 약 85초

그런데 이 한 줄에 함정이 하나 있습니다. 제가 참고 코드로 직접 돌려 본 실험입니다.

대공 표적을 고도 300 미터에서 남쪽으로 60초 직진시켰습니다. 두 가지 방법으로요. A 는 처음에 구한 ECEF 속도를 60초 내내 그대로 쓴 겁니다. B 는 매 스텝 표적의 현재 위경도로 수평면을 다시 잡고, 그 위에서 속도 방향을 다시 만든 겁니다.

결과가 다릅니다. A 는 60초 뒤 고도가 311.3 미터입니다. 11 미터 떠올랐습니다. B 는 300.0 미터, 그대로입니다.

왜 떠오르느냐. 지구가 둥글기 때문입니다. 처음 자리의 수평면은 그 자리의 접선입니다. 접선 방향으로 12 킬로미터를 쭉 가면 땅은 그 아래로 휘어 내려갑니다. 얼마나 내려가느냐면 거리 제곱 나누기 2R, 12,000 제곱 나누기 2 곱하기 6,371,000, 계산하면 11.3 미터입니다. 실험값과 딱 맞습니다.

그래서 매 스텝 위경도로 되돌리는 마지막 단계가 그냥 출력용이 아닙니다. 다음 스텝의 수평면을 잡는 기준을 만드는 일입니다. 위경도를 구해야 NED 축이 나오고, NED 축이 있어야 속도 방향이 나옵니다. 이 순서를 지키면 수평 비행은 수평으로 유지됩니다.

지난 과제 마지막에 지구는 둥글다고 정리했었는데, 이번에도 같은 얘기가 다른 모습으로 나온 겁니다.

01. 한 스텝의 순서 — 다섯 단계

표적 하나가 0.1초 뒤로 가는 일. 이 순서를 600번, 표적 수만큼 반복한다

표적 하나의 한 스텝 (0.1 초)

왼쪽부터 순서대로. ① ② 는 기동이 없으면 건너뛰고, ③ ④ ⑤ 는 매 스텝 반드시 한다.



① 에 필요한 것
기동 목록, 지금 시각 t

② 에 필요한 것
각속도 $\omega = n \cdot g / V, \Delta t$

③ 에 필요한 것
자세각 3개, 표적 위경도

④ 에 필요한 것
ECEF 위치-속도, $\Delta t = 0.1 s$

⑤ 에 필요한 것
ECEF → LLA 변환 함수

표적 10개 × 600 스텝 = 6,000번. 순서가 중요한 건 ③ ④ ⑤ — 자세가 정해져야 속도, 속도가 있어야 위치, 위치가 있어야 새 수평면

12

발표 대본

원리 · 약 70초

지금까지 나온 걸 한 스텝의 순서로 정리하면 다섯 단계입니다. 표적 하나가 0.1초 뒤로 가는 일입니다.

하나, 기동 판단. 지금 시각이 어느 기동 구간에 들어 있는지 봅니다. 둘, 자세 갱신. 기동 중이면 각속도 곱하기 0.1초만큼 자세각을 돌립니다. 이 둘은 다음 장에서 자세히 하겠습니다.

셋, 속도 변환. 동체에서 NED, NED에서 ECEF, 회전 두 번. 넷, 위치 적분. ECEF에서 속도 곱하기 0.1초를 더합니다. 다섯, 위경도 변환. 기록도 하고, 다음 스텝의 수평면 기준도 만듭니다.

그리고 시각을 0.1초 올려서 처음으로 돌아갑니다. 60초까지 600번입니다. 표적이 열 개면 표적마다 이 다섯 단계를 도니까 6,000번이 됩니다. 컴퓨터한테는 아무것도 아닌 양입니다.

순서가 중요한 건 셋, 넷, 다섯입니다. 자세가 정해져야 속도가 나오고, 속도가 있어야 위치를 옮기고, 위치를 옮겨야 새 수평면이 나옵니다. 하나, 둘은 기동이 없는 스텝에서는 건너뛰니다.

02

기동 모델

- 1 기동이란 — 하중배수 g
- 2 원운동 공식 — 각속도와 선회 반경
- 3 세 축 — Roll · Yaw · Pitch
- 4 기동 시간표 — StartTime · EndTime

약속 : 기동 중에도 속력은 바뀌지 않는다. 방향만 바뀐다.

13

발표 대본

장절 · 약 25초

둘째 장은 기동입니다. 표적이 방향을 바꾸는 걸 어떻게 숫자로 만드느냐입니다. 과제가 준 자료는 네 개입니다. 몇 g 인가, 어느 축인가, 언제 시작하고 언제 끝나는가.

여기서 약속 하나. 기동 중에도 속력은 바뀌지 않는다고 놓습니다. 방향만 바뀝니다. 그래야 초속 200 미터라는 값 하나로 계속 갈 수 있습니다.

02. 기동이란 — 하중배수 g

과제의 Gravity Value 는 "몇 g 로 도는가". 속력은 그대로, 방향만 바뀐다



14

발표 대본

기동 · 약 80초

과제에 Gravity Value 라는 항목이 있습니다. 이게 하중배수, 흔히 몇 g 라고 부르는 그 숫자입니다.

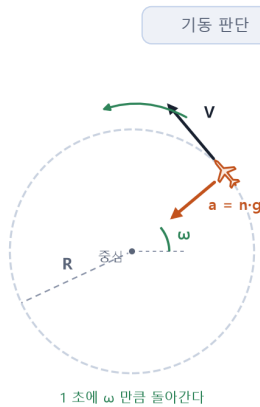
왼쪽 그림, 커브를 도는 자동차입니다. 속력은 그대로인데 방향이 계속 바뀝니다. 방향이 바뀌려면 안쪽으로 당기는 가속도가 있어야 합니다. 이 가속도의 크기를 중력가속도 g 의 몇 배인가로 적은 게 하중배수 n 입니다. 가속도가 n 곱하기 g 입니다.

오른쪽이 감을 잡기 위한 눈금입니다. 가만히 서 있으면 1 g. 승용차가 커브를 돌 때 0.3 g 정도. 롤러코스터 바닥이나 전투기 급선회가 3 g. 에어쇼 기동이 5 g 인데 이쯤 되면 시야가 흐려지기 시작합니다. 전투기 한계가 9 g 고 조종사가 대G복을 입어야 버팁니다.

중요한 성질이 하나 있습니다. 이 가속도는 속도에 수직입니다. 속도 방향으로의 밀지 않고 옆으로만 당깁니다. 그래서 속력은 안 변하고 방향만 변합니다. 아까 약속한 게 물리적으로도 맞는 얘기인 겁니다.

그러면 이 숫자 하나로 표적이 얼마나 빨리 도는지가 정해집니다. 다음 장에서 그 공식을 보겠습니다.

02. 원운동 공식 — 각속도와 선회 반경

고등학교 물리의 등속 원운동 그대로, 같은 g 라도 느린 표적이 훨씬 빨리 돈다

등속 원운동 공식

구심가속도
 $a = V^2 / R = n \cdot g$ 선회 반경
 $R = V^2 / (n \cdot g)$ 각속도
 $\omega = V / R = n \cdot g / V$ $g = 9.80665 \text{ m/s}^2$
 V 는 변하지 않는다

표적	하중배수	각속도	선회 반경	비고
대공 200 m/s	1 g	2.81 °/s	4,079 m	180° 에 64 s
대공 200 m/s	3 g	8.43 °/s	1,360 m	180° 에 21 s
대공 200 m/s	5 g	14.05 °/s	816 m	180° 에 13 s
대함 30 m/s	0.1 g	1.87 °/s	918 m	90° 에 48 s

각속도가 속력에 반비례한다

 $\omega = n \cdot g / V$. 배는 작은 g 로도 잘 돌고, 빠른 항공기는 큰 g 가 있어야 돈다. 그래서 배 기동에는 0.1 g 같은 값을 넣어야 그럴듯하다.프로그램에서는 한 줄 : $\omega = n \cdot g / V$ 를 구해 놓고, 매 스텝 자세각에 $\omega \times 0.1 \text{ s}$ 를 더한다 (3 g 면 한 스텝에 0.84°)

15

발표 대본

기동 · 약 80초

고등학교 물리 등속 원운동 그대로입니다.

반경 R 인 원을 속력 V 로 돌면 안쪽으로 당기는 가속도가 V 제곱 나누기 R 입니다. 이게 n 곱하기 g 와 같습니다. 이 식을 R 에 대해 풀면 선회 반경이 V 제곱 나누기 $n \cdot g$. 각속도, 그러니까 1초에 몇 도 도느냐는 V 나누기 R 인데, 정리하면 $n \cdot g$ 나누기 V 입니다.

오른쪽 표가 실제 숫자입니다. 초속 200 미터 대공 표적이 1 g 로 돌면 1초에 2.8도, 반경이 4 킬로미터입니다. 180도 돌아서는 데 64초, 우리 시뮬레이션 시간을 넘깁니다. 3 g 면 1초에 8.4도, 반경 1.36 킬로미터, 반바퀴에 21초. 5 g 면 14도, 반경 816 미터. 초속 30 미터 배는 0.1 g 만 돼도 1초에 1.9도, 반경 918 미터입니다.

여기서 알 수 있는 게, 같은 g 라도 느린 표적이 훨씬 빨리 돕니다. 각속도가 속력에 반비례하니까요. 그래서 배 기동에는 0.1 g 같은 작은 값을 넣어야 그럴듯한 궤적이 나옵니다.

프로그램에서 하는 일은 각속도 한 줄입니다. $n \cdot g$ 나누기 V 를 구해 놓고, 매 스텝 자세각에 각속도 곱하기 0.1초를 더합니다. 3 g 면 한 스텝에 0.84도씩입니다.

02. 세 축 — Roll · Yaw · Pitch

TurnType 은 동체의 어느 축을 돌리는가. Roll 만 속도 방향을 바꾸지 않는다

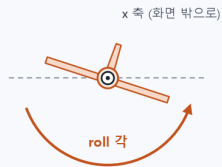
기동 판단

자세 갱신

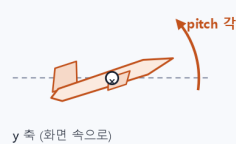
속도 변환

위치 적분

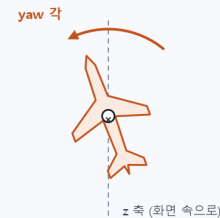
LLA 변환

Roll (x 축 · 앞뒤 축)
뒤에서 본 그림 · 날개가 기운다

속도 방향은 그대로

Pitch (y 축 · 날개 축)
동에서 본 그림 · 기수가 들린다

상승 · 강하

Yaw (z 축 · 위아래 축)
위에서 본 그림 · 기수가 돌아간다

좌우 선회

TurnType	축	효과
0	—	직진
1 Roll	x (앞뒤)	자세만 기울
2 Yaw	z (위아래)	좌우 선회
3 Pitch	y (날개)	상승 · 강하

Roll 은 준비 동작

롤 90° 뒤에 피치로 당기면 위가 아니라 옆으로 돈다.

실제 비행기의 선회 : 기울이고, 당긴다.

축을 돌린 뒤 속도가 어느 쪽인지는 따로 계산하지 않는다 — 자세각만 바뀌 두면 9장의 회전 두 번이 새 속도 방향을 만든다
세 축의 각속도는 모두 $\omega = n \cdot g / V$ 로 같게 둔다 (정의의 문제, 발표에서 명시).

16

발표 대본

기동 · 약 85초

과제의 TurnType 이 어느 축을 돌리느냐입니다. 0 은 안 돌리기, 1 이 롤, 2 가 요, 3 이 피치입니다.

가운데 그림부터 보겠습니다. 피치는 날개 축, 그러니까 동체 y 축을 중심으로 기수가 들리는 겁니다. 이러면 상승하거나 강하합니다. 오른쪽, 요는 위아래 축, 동체 z 축을 중심으로 기수가 좌우로 돌아가는 겁니다. 이게 수평 선회입니다. 배가 방향을 바꾸는 것도 이겁니다.

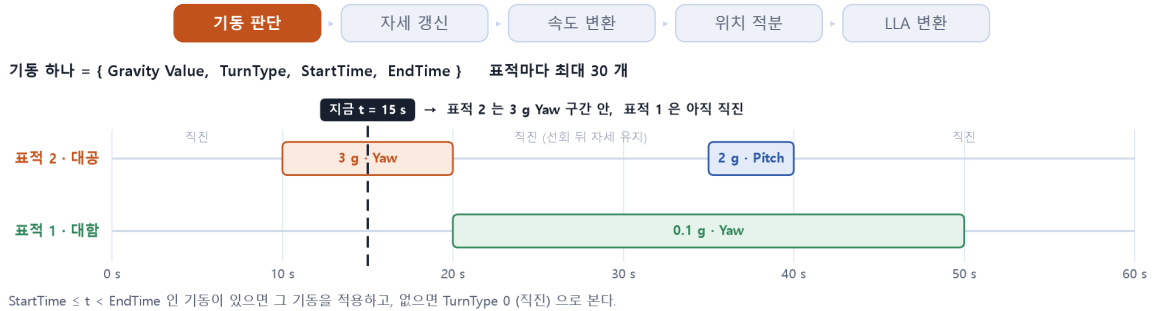
왼쪽 롤이 좀 특이합니다. 앞뒤 축, 그러니까 속도 방향 축을 중심으로 날개가 기우는 겁니다. 축이 속도 방향과 같으니까 롤을 해도 속도 방향은 안 바뀝니다. 그러면 뭐가 바뀌느냐. 자세가 바뀝니다. 그리고 그 뒤에 오는 기동의 방향이 바뀝니다. 오른쪽으로 90도 롤을 한 다음에 피치로 당기면 위로 올라가는 게 아니라 오른쪽으로 돕니다. 실제 비행기가 선회하는 방식이 그겁니다. 먼저 기울이고, 그다음 당깁니다.

그래서 이번 모델에서 롤은 궤적을 직접 바꾸지는 않지만 다음 기동을 위한 준비 동작으로 의미가 있습니다. 세 축의 각속도는 전부 아까 공식 $n \cdot g$ 나누기 V 로 같게 두겠습니다.

축을 돌린 다음 속도가 어느 쪽인지는 따로 계산할 필요가 없습니다. 자세각만 바뀌 두면 9장에서 본 회전 두 번이 자동으로 새 속도 방향을 만들어 줍니다.

02. 기동 시간표 — StartTime · EndTime

기동 하나 = { g, TurnType, StartTime, EndTime }. 표적마다 최대 30개



구간 안이면 적용, 밖이면 직진

StartTime ≤ t < EndTime.
없으면 TurnType 0 으로 본다.

정해 둘 규칙 셋

부호 : $n < 0$ 이면 반대 방향. 겹침 : 먼저 적힌 것.
끝난 뒤 : 그 순간 자세를 유지하고 직진.

최대 30개

배열 30칸 + 개수 하나.
과제 조건에는 기동 목록이 없다 → 예시를 우리가 정한다.

예시 기동을 넣지 않으면 직진 두 개만 그려진다 — 기동 30개짜리 구조체의 의미가 보이도록 발표용 시나리오를 하나 더 만든다

17

발표 대본

기동 · 약 75초

기동 하나는 네 값으로 적습니다. 몇 g, 어느 축, 시작 시각, 끝 시각. 이것 표적마다 최대 30개까지 배열에 담습니다.

그림이 예시 시간표입니다. 대공 표적은 10초부터 20초까지 3 g 요, 35초부터 40초까지 2 g 피치. 대함 표적은 20초부터 50초까지 0.1 g 요. 나머지 구간은 직진입니다.

매 스텝 첫 단계가 이것 보는 겁니다. 지금 시각이 15초면, 대공 표적은 첫 번째 기동 구간 안에 있으니 3 g 요를 적용하고, 대함 표적은 아직 아무 구간에도 안 들어 있으니 직진입니다. 조건은 시작 시각 이하, 끝 시각 미만입니다.

정해 뒤에 할 규칙이 셋 있습니다. 첫째, 부호. g 값이 음수면 반대 방향으로 둡니다. 오른쪽 선회와 왼쪽 선회를 하나의 값으로 구분하는 겁니다. 둘째, 겹침. 두 기동의 시간이 겹치면 먼저 적힌 걸 씁니다. 셋째, 끝난 뒤. 기동이 끝나면 그 순간의 자세를 그대로 유지하고 직진합니다. 원래 방향으로 돌아가지 않습니다.

한 가지 짚어 둘 게, 과제 조건에는 기동 목록이 없습니다. 초기값만 있습니다. 그래서 발표용 예시 기동은 우리가 정해서 넣어야 합니다. 안 그러면 직진 두 개만 그려지고, 기동 30개짜리 구조체를 만든 의미가 안 보입니다.

03

구조체와
시뮬레이션 설계

- 1 표적 구조체 — 표적 카드 한 장
- 2 시나리오와 전체 흐름
- 3 시뮬레이션 조건 — 플랫폼 하나, 표적 둘
- 4 예상 결과 — 정면 접근과 가로지르기
- 5 검증 계획 — 무엇을 확인할 것인가
- 6 결과 출력과 그림
- 7 다음 단계 — 구현 순서

과제 기타 사항 : 구조체 정의 설명 · 궤적 모의 방법 설명 · 결과 설명 · 제공 코드 이용

18

발표 대본

장절 · 약 25초

셋째 장은 설계입니다. 앞에서 본 원리를 프로그램 모양으로 옮깁니다. 표적 구조체를 어떻게 잡을지, 전체 흐름이 어떻게 되는지, 과제 조건을 어떻게 넣을지, 결과가 어떻게 나올지, 그리고 그걸 어떻게 검증할지까지 일곱 장입니다.

과제 기타 사항에 구조체 정의 설명과 궤적 모의 방법 설명이 요구되어 있는데, 그 두 가지가 이 장입니다.

03. 표적 구조체 — 표적 카드 한 장

입력 칸은 사람이 적고, 상태 칸은 프로그램이 갱신한다. 구조체 모양에 과제 요구가 보이도록

표적 카드 #2

입력 (사람이 적는 칸)

초기 위치 위경도 · 고도로
32.12°N 126.0°E 300 m

동체 속도 앞 방향 하나로
200 m/s

초기 자세
roll 0° pitch 0° yaw 180°

기동 목록 (최대 30 개)

g	Type	Start	End
3	Yaw	10	20
2	Pitch	35	40
...			

상태 (프로그램이 갱신)

ECEF 위치
x, y, z [m]

ECEF 속도
vx, vy, vz [m/s]

현재 자세
roll, pitch, yaw

현재 위경도
출력용 · 매 스텝 변환

0.1 s 매 스텝 갱신 · 600 번
입력 칸은 건드리지 않는다

초기화
한 번

C 구조체 (ST_접두사 · typedef, 코딩 규칙대로)

```
typedef struct {
    FLOAT64 dGravity; // 하중배수 [g], 부호 = 방향
    INT32 nTurnType; // 0 없음 1 Roll 2 Yaw 3 Pitch
    FLOAT64 dStartTime; // [s]
    FLOAT64 dEndTime; // [s]
} ST_Maneuver;

typedef struct {
    /* 입력 - 사람이 적는 칸 */
    ST_Lla stInitLla; // 위도·경도·고도
    FLOAT64 dVheading; // 동체 속도 [m/s]
    ST_Attitude stInitAtt; // roll · pitch · yaw
    INT32 nManeuverCnt; // <= MAX_MANEUVER (30)
    ST_Maneuver astManeuver[MAX_MANEUVER];
    /* 상태 - 프로그램이 갱신 */
    ST_Vec3 stPosEcef; // ECEF 위치 [m]
    ST_Vec3 stVelEcef; // ECEF 속도 [m/s]
    ST_Attitude stAtt; // 현재 자세
    ST_Lla stLla; // 현재 위경도 (출력용)
} ST_Target;
```

위 다섯 줄이 입력, 아래 네 줄이 상태 — “초기값은 위경도, 갱신은 ECEF” 라는 과제 요구가 구조체 모양에 그대로 드러난다

19

발표 대본

설계 · 약 90초

표적 하나를 카드 한 장이라고 생각하면 편합니다. 왼쪽 그림이 그 카드입니다.

카드는 두 칸으로 나뉩니다. 왼쪽이 입력 칸, 사람이 적는 자리입니다. 처음 위치를 위경도와 고도로, 속력을 동체 앞 방향 하나로, 처음 자세를 롤 피치 요로, 그리고 기동 목록을 최대 30개까지. 과제가 요구한 입력 형식이 그대로 칸이 됩니다.

오른쪽이 상태 칸, 프로그램이 갱신하는 자리입니다. ECEF 위치, ECEF 속도, 현재 자세, 그리고 출력용 현재 위경도. 과제가 ECEF에서 갱신하라고 한 그 값들입니다.

처음에 한 번 입력 칸을 읽어서 상태 칸을 채우고, 그 뒤 600번은 상태 칸만 고칩니다. 입력 칸은 건드리지 않습니다. 그래서 같은 시나리오를 다시 돌리고 싶으면 입력 칸만 있으면 됩니다.

오른쪽이 이걸 C 구조체로 적은 모습입니다. 기동 하나가 ST_Maneuver, 표적 하나가 ST_Target 입니다. 이름은 코딩 규칙대로 구조체는 ST 접두사, typedef 로 선언했습니다. 위쪽 다섯 줄이 입력, 아래 네 줄이 상태입니다. 구조체 모양만 봐도 초기값은 위경도, 갱신은 ECEF 라는 과제 요구가 보이도록 나눴습니다.

03. 시나리오와 전체 흐름

카드가 여러 장 모이면 시나리오, 플랫폼도 속도 0 인 카드 한 장으로 둔다

프로그램 전체 흐름

플랫폼도 속도 0 인 표적 카드로 두면, 갱신 함수 하나로 전부 처리된다.



루프 안 5 단계: 기동 판단 → 자세 갱신 → 속도 변환 → 위치 적분 → LLA 변환 (12 장)

시나리오 구조체

```

typedef struct {
    INT32    nTargetCnt;           // ≤ MAX_TARGET (10)
    ST_Target astTarget[MAX_TARGET];
    ST_Target stPlatform;         // 속도 0, 기동 0개
    FLOAT64  dSimTime;            // 60 s
    FLOAT64  dDt;                 // 0.1 s
} ST_Scenario;
  
```

플랫폼도 표적 카드로

속도 0, 기동 0개인 표적일 뿐. 갱신 함수가 하나로 통일되고, 플랫폼이 움직이는 시나리오가 와도 카드 값만 바뀌면 된다.

CSV 한 줄 = 한 표적의 한 스텝

t, id, lat, lon, alt (+ 플랫폼 기준 N·E·D, 거리, 고각)
60초 × 10 Hz × 표적 수. 그림은 파이썬으로 따로 그린다.

20

발표 대본

설계 · 약 70초

카드가 여러 장 모이면 시나리오입니다. 아래 구조체가 그겁니다. 표적 카드가 최대 10장, 플랫폼 카드 한 장, 그리고 시뮬레이션 시간 60초와 간격 0.1초.

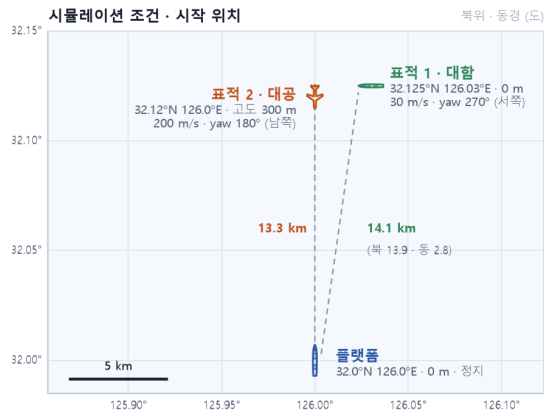
플랫폼도 표적 카드로 만든 게 요령입니다. 이번 과제에서 플랫폼은 속도 0, 기동 0개인 표적일 뿐입니다. 그러면 위치를 갱신하는 함수가 하나만 있으면 되고, 나중에 플랫폼이 움직이는 시나리오가 와도 카드 값만 바꾸면 됩니다.

위 그림이 프로그램 전체 흐름입니다. 시나리오를 읽고, 초기화에서 위경도를 ECEF로 바꾸고 자세와 속력을 복사합니다. 그다음 루프 600번. 표적마다 12장의 다섯 단계를 돕니다. 매 스텝 CSV에 한 줄씩 적습니다. 시각, 표적 번호, 위도, 경도, 고도. 여기에 플랫폼 기준 거리와 고각도 같이 적어 두면 나중에 레이다 입장에서 보는 그림을 그릴 수 있습니다.

마지막이 그림입니다. CSV를 파이썬으로 읽어서 세 장을 그립니다. 그건 24장에서 보겠습니다.

03. 시뮬레이션 조건 — 플랫폼 하나, 표적 둘

대공 표적은 정면으로 접근하고, 대함 표적은 정북 방향을 가로지른다



항목	플랫폼	대함 표적 (1)	대공 표적 (2)
위도 · 경도	32.0°N 126.0°E	32.125°N 126.03°E	32.12°N 126.0°E
고도	0 m	0 m	300 m
속력 (동체 앞)	0 m/s	30 m/s	200 m/s
자세각 (r · y · p)	—	0° · 270° · 0° (서쪽)	0° · 180° · 0° (남쪽)
시간	60 s, 간격 0.1 s (600 스텝)		

플랫폼 기준으로 바꿔 보면 (참고 코드로 계산)

대공 : 정북 13.3 km, 고각 1.2° → 정확히 우리 쪽으로 온다 (정면 접근)

대함 : 북 13.9 km · 동 2.8 km, 거리 14.1 km → 서쪽으로 가며 정북을 가로지른다

두 표적의 출발점은 550 m · 2.8 km 차이로 아주 가깝다

같은 자리에서 출발해 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 — 그림에서 대비가 분명하도록 짜인 조건

21

발표 대본

설계 · 약 70초

과제가 준 조건을 지도 위에 놓은 겁니다.

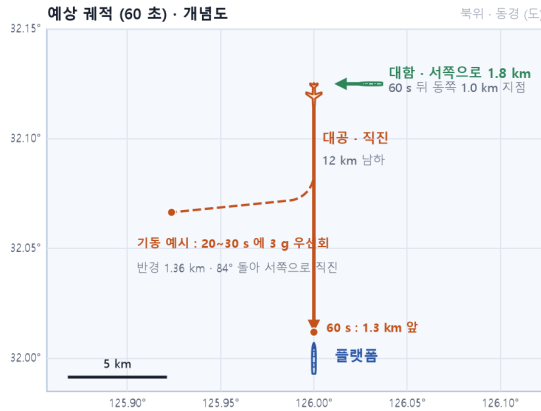
플랫폼이 북위 32도, 동경 126도, 고도 0, 정지. 대공 표적이 북위 32.12도, 같은 경도, 고도 300 미터. 초속 200 미터, 요 180도니까 정남쪽을 봅니다. 대함 표적이 북위 32.125도, 동경 126.03도, 고도 0. 초속 30 미터, 요 270도니까 정서쪽입니다.

이걸 플랫폼 기준으로 바꿔 보면 상황이 보입니다. 참고 코드로 계산한 값입니다. 대공 표적은 정북 13.3 킬로미터, 고각 1.2도. 정확히 우리 쪽으로 날아옵니다. 정면 접근 시나리오입니다. 대함 표적은 북쪽 13.9 킬로미터, 동쪽 2.8 킬로미터, 거리로 14.1 킬로미터. 서쪽으로 가니까 우리 정북 방향을 가로지르게 됩니다.

두 표적이 지도에서 아주 가까이 있는 게 보이실 겁니다. 대함 표적이 대공 표적 출발점에서 북쪽으로 550 미터, 동쪽으로 2.8 킬로미터입니다. 일부러 이렇게 준 것 같습니다. 같은 자리에서 출발해서 하나는 남쪽으로 빠르게, 하나는 서쪽으로 천천히 가면 그림에서 대비가 분명하니까요.

03. 예상 결과 — 정면 접근과 가로지르기

코드를 돌리기 전에 결과가 어떻게 나와야 하는지 먼저 그려 둔다



대공 — 정면 접근

13.3 km → 1.3 km, 고각 1.2° → 13.4°. 60초 뒤엔 거의 머리 위. 고도는 300 m 그대로여야 한다 (11장의 곡률 처리가 맞았다면).

대함 — 가로지르기

서쪽으로 1.8 km. 대공 표적 출발점 바로 북쪽을 지나 동쪽 1.0 km 지점에서 끝.

기동 예시를 넣으면

대공 표적에 20 ~ 30 s 동안 3 g 우선회 : 10초에 84° 돌아 서쪽을 보고, 그 뒤 직진. 반경 1.36 km 원호가 그려진다. 조건대로 돌린 것과 기동을 넣은 것을 나란히 보여 준다.

결과가 어때야 하는지 미리 알아야, 나온 결과가 맞는지 판단할 수 있다 → 다음 장의 검증 계획

22

발표 대본

설계 · 약 75초

코드를 돌리기 전에 결과가 어떻게 나와야 하는지 미리 그려 봤습니다. 이래야 나중에 결과가 맞는지 판단할 수 있습니다.

대공 표적. 정남으로 초속 200 미터, 60초면 12 킬로미터. 13.3 킬로미터 앞에서 출발했으니 60초 뒤엔 1.3 킬로미터 앞입니다. 고각은 1.2도에서 13.4도로 올라갑니다. 거의 머리 위까지 오는 겁니다. 고도는 300 미터 그대로여야 합니다. 11장의 곡률 처리가 제대로 됐으면요.

대함 표적. 서쪽으로 초속 30 미터, 60초면 1.8 킬로미터. 대공 표적 출발점 바로 북쪽을 지나서 동쪽 1 킬로미터 지점에서 끝납니다.

점선이 기동 예시입니다. 대공 표적에 20초부터 30초까지 3 g 우선회를 넣으면, 10초 동안 84도를 돌아서 서쪽을 보게 되고 그 뒤로 직진합니다. 반경 1.36 킬로미터짜리 원호가 그려집니다. 이 정도면 기동 기능이 한눈에 보입니다.

직진 두 개만 그리면 그림이 심심합니다. 발표에서는 조건대로 돌린 것과, 기동을 넣은 것을 나란히 보여 드리려고 합니다.

03. 검증 계획 — 무엇을 확인할 것인가

좌표변환은 틀려도 그럴듯한 값이 나온다 (과제 2의 교훈). 점이 600개면 눈으로는 더 못 잡는다

항목	어떻게 확인하나	기대값	참고 코드로 미리 확인한 값
① 직진 거리	속력 × 시간과 비교	$200 \times 60 = 12,000 \text{ m}$	13,307 m → 1,307 m (12,000 m)
② 속력 보존	매 스텝 $ v_{\text{ecef}} $ 와 입력 속력 비교	200.000 m/s	200.000 m/s
③ 고도 유지	수평 비행 60초 뒤 고도	$300 \text{ m} \pm 0.1$	300.02 m
④ 선회 반경	궤적에서 반경을 재서 $V^2 / (ng)$ 와 비교	3 g : 1,360 m	코드 완성 후
⑤ 위경도 왕복	LLA → ECEF → LLA 오차	1 mm 미만	0.03 mm

왜 미리 정하나

코드가 내놓을 값을 먼저 알고 있어야 코드가 맞는지 알 수 있다.
 ①②③⑤는 참고 코드만으로 이미 확인했고, ④는 기동 코드가 붙으면 쟤다.

한 번 틀리면 바로 드러나는 실수들

경도·위도 인자 순서 바꿈 → 북위 54° 동경 212°. 속도에 플랫폼 위치 더함 → 6,372 km/s.
 수평면을 한 번만 잡음 → 고도 311 m.

이 다섯 칸이 채워지면 궤적이 "그림" 이 아니라 "결과" 가 된다

23

발표 대본

설계 · 약 75초

지난 과제에서 얻은 교훈이 하나 있습니다. 좌표변환은 틀려도 그럴듯한 값이 나온다. 이번엔 점이 600개라 눈으로는 더 못 잡습니다. 그래서 확인 항목을 미리 정했습니다.

첫째, 직진 거리. 속력 곱하기 시간이니깐 대공 표적은 정확히 12 킬로미터를 가야 합니다. 참고 코드로 미리 돌려 보니 13,307 미터에서 1,307 미터로, 딱 12,000 입니다.

둘째, 속력 보존. 매 스텝 ECEF 속도의 크기가 입력한 속력과 같아야 합니다. 회전만 했으니까요. 200.000 이 나왔습니다.

셋째, 고도 유지. 수평 비행이면 60초 뒤에도 300 미터여야 합니다. 300.02 미터, 2 센티미터 차이입니다.

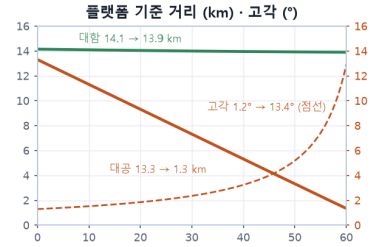
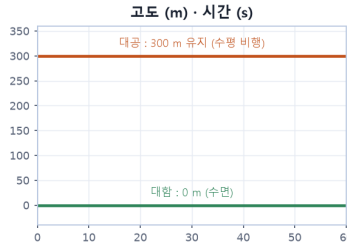
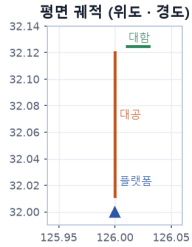
넷째, 선회 반경. 기동을 넣고 궤적에서 반경을 재서 V 제곱 나누기 ng 와 비교합니다. 3 g 면 1,360 미터가 나와야 합니다.

다섯째, 위경도 왕복. 위경도를 ECEF로 갔다가 돌아오면 같은 값이어야 합니다. 참고 코드는 반복을 다섯 번 고정으로 하는데, 오차가 0.03 밀리미터라 충분합니다.

이 다섯 개가 표로 채워지면 궤적이 그림이 아니라 결과가 됩니다.

03. 결과 출력과 그림 — 위경도로 적고 세 장으로 그린다

직진만 가정하고 미리 그려 본 모양. 실제 시뮬레이션 결과로 갈아 끼울 자리



직진만 가정하고 계산한 예상 모양이다. 실제 시뮬레이션 결과로 갈아 끼울 자리.

① 평면 궤적

경도 가로, 위도 세로. 축 비율을 $1 / \cos 32^\circ$ 로 맞춰야 선이 원이 찌그러진 타원으로 보이지 않는다.

② 고도 - 시간

대공 300 m, 대함 0 m에서 평평해야 한다. 곡률 처리(11장)의 검증 그림이기도 하다.

③ 거리 · 고각 - 시간

플랫폼 기준. 레이더가 실제로 보게 될 값. 대공 13.3 → 1.3 km, 고각 1.2° → 13.4°.

CSV (t, id, lat, lon, alt, ...) → Python matplotlib → 그림 세 장. 과제 요구 (시뮬레이션 결과를 LLA로 출력 후 그림으로 표현)에 대응

24

발표 대본

설계 · 약 65초

결과는 위경도로 적고 그림 세 장으로 보입니다. 지금 보시는 건 직진만 가정하고 미리 그려 본 모양이고, 실제 결과로 갈아 끼울 자리입니다.

첫째, 평면 궤적. 경도를 가로, 위도를 세로로 놓고 두 표적과 플랫폼을 찍습니다. 여기서 조심할 게 하나 있습니다. 7장에서 본 대로 위도 1도와 경도 1도의 길이가 다르니까, 축 비율을 1 나누기 코사인 32도로 맞춰야 합니다. 안 그러면 선이 원이 찌그러진 타원으로 보입니다.

둘째, 고도와 시간. 대공은 300에서 평평하게, 대함은 0에서 평평하게 나와야 합니다. 이 그림이 곡률 처리의 검증이기도 합니다. 조금이라도 기울면 11장의 문제가 남아 있는 겁니다.

셋째, 플랫폼 기준 거리와 고각. 대공 표적 거리가 13.3에서 1.3킬로미터로 떨어지고 고각이 1.2도에서 13.4도로 올라갑니다. 레이더가 실제로 보게 될 값이 이겁니다. 대함은 거리가 14.1에서 13.9로 거의 안 변합니다. 옆으로 지나가니까요.

CSV를 파이썬 matplotlib로 읽어서 그림니다.

03. 다음 단계 — 구현 순서

오늘 본 순서 그대로 코드로 옮긴다. 손계산 검산값을 먼저 만들어 둔다

- 1 구조체 셋 정의 : ST_Maneuver · ST_Target · ST_Scenario (코딩 규칙대로 ST_ 접두사, typedef)
- 2 초기화 : 위경도 → ECEF (참고 코드 f_Trans_Lla_To_Ecef, 경도가 첫째 인자)
- 3 속도 변환 함수 : 동체 → NED → ECEF, 회전 두 번, 플랫폼 위치 인자는 0
- 4 기동 판단과 자세 갱신 : 구간 검사, $\omega = n \cdot g / V$, 자세각 += $\omega \times \Delta t$
- 5 루프와 CSV : 60 s / 0.1 s, 표적마다 5단계, 한 스텝에 한 줄
- 6 그림 세 장 : Python matplotlib, 축 비율 1 / cos 32°
- 7 검증 표 채우기 : 거리 · 속력 · 고도 · 반경 · 왕복

일정 (나흘)

- 1일차 수식 손계산 · 검산값
- 2일차 구조체 · 루프 · 속도 변환
- 3일차 그림 · 검증 표
- 4일차 발표 자료 (결과편)

손계산을 먼저 하는 이유

코드가 내놓을 값을 미리 알고 있어야 코드가 맞는지 알 수 있다 (23장).

다음 발표 : 코드와 실행 결과, 검증 표, 기동 시나리오 비교

25

발표 대본

설계 · 약 55초

구현 순서입니다. 오늘 본 순서 그대로 갑니다.

하나, 구조체 셋을 정의합니다. 기동, 표적, 시나리오. 둘, 초기화. 위경도를 ECEF로 바꾸는 함수는 참고 코드에 있습니다. 경도가 먼저 오는 인자 순서만 조심하면 됩니다. 셋, 속도 변환 함수. 회전 두 번, 플랫폼 위치는 0. 넷, 기동 판단과 자세 갱신. 다섯, 루프와 CSV. 여섯, 파이썬 그림 세 장. 일곱, 검증 표를 채웁니다.

일정은 나흘로 잡았습니다. 첫날 수식을 손으로 계산해서 검산값을 만들어 두고, 둘째 날 코드, 셋째 날 그림과 검증, 넷째 날 발표 자료입니다.

손계산을 먼저 하는 이유는 23장 때문입니다. 코드가 내놓을 값을 미리 알고 있어야 코드가 맞는지 알 수 있습니다.

정리

- 1 과제 3 = 과제 2의 변환 체인 + 시간 루프 + 기동 모델. 좌표변환 함수 넷은 그대로 쓴다
- 2 위치는 회전 + 평행이동, 속도는 회전만. 속도에 위치를 더하면 안 된다
- 3 새 위치 = 지금 위치 + 속도 $\times$ 0.1 초. ECEF 에서 더한다
- 4 지구는 둥글다. 매 스텝 표적 자리의 수평면을 다시 잡아야 고도가 유지된다
- 5 기동은 원운동. $\omega = n \cdot g / V$, $R = V^2 / (n \cdot g)$. 속력은 그대로
- 6 표적 카드 = 입력 칸 + 상태 칸. 플랫폼도 카드 한 장
- 7 검증 없는 궤적은 그림일 뿐. 거리 · 속력 · 고도 · 반경으로 확인한다

질문 환영합니다

다음 발표 : 코드와 실행 결과

코드 · 문서 : [radar-sw-study / Part3](#)

26

발표 대본

정리 · 약 60초

일곱 가지로 정리하겠습니다.

하나, 과제 3은 과제 2의 변환 체인에 시간 루프와 기동 모델을 얹은 것입니다. 둘, 위치는 회전과 평행이동, 속도는 회전만입니다. 속도에 위치를 더하면 안 됩니다. 셋, 새 위치는 지금 위치 더하기 속도 곱하기 0.1초, ECEF에서 더합니다. 넷, 지구는 둥글다. 매 스텝 수평면을 다시 잡아야 고도가 유지됩니다. 다섯, 기동은 원운동입니다. 각속도는 $n \cdot g$ 나누기 V , 반경은 V 제곱 나누기 $n \cdot g$. 속력은 그대로입니다. 여섯, 표적 카드는 입력 칸과 상태 칸으로 나눕니다. 플랫폼도 카드 한 장입니다. 일곱, 검증 없는 궤적은 그림일 뿐입니다. 거리, 속력, 고도, 반경으로 확인합니다.

이상입니다. 다음 발표에서 코드와 실제 결과를 가지고 오겠습니다. 질문 환영합니다.

부록 A. 참고 코드의 함수와 함정

제공된 CoordinateTransform.c 에서 이번에 쓰는 것과, 컴파일해서 확인한 주의점

함수	이번 과제의 용도	주의
f_Trans_Lla_To_Ecef (Lon, Lat, Alt)	초기 위치 입력	경도가 첫째 인자. 바꾸면 북위 54° 동경 212°
f_Trans_Ecef_To_Lla (X, Y, Z)	매 스텝 결과 출력 + 다음 수평면 기준	5회 고정 반복. 우리 위치에서 오차 0.03 mm
f_Trans_Body_To_Ned (... , Roll, Yaw, Pitch)	동체 속력 → NED 속도	인자 순서가 Roll, Yaw, Pitch
f_Trans_Ned_To_Ecef (... , Pf_X, Pf_Y, Pf_Z, Lat, Lon)	NED 속도 → ECEF 속도	속도에는 Pf_X-Y-Z = 0. 아니면 6,372 km/s
f_Trans_Ecef_To_Ned (...)	플랫폼 기준 거리 · 고각 (결과 그림용)	각도는 전부 라디안. DEG2RAD 매크로 직접 정의
Ant 계열 4개 · Enu 계열 2개 · PfCompensation	사용 안 함	표적을 만드는 쪽이라 안테나 좌표계가 없다

Body → NED 행렬은 Rz(yaw) · Ry(pitch) · Rx(roll) 과 소수점 6자리까지 일치 — 과제 2 의 coord_frames.c 와 같은 규약
 그 밖에 : 한글 주석이 UTF-8 이라 MSVC 에서 /utf-8 옵션 필요. matrixCalLib 은 9×9 고정 행렬 (과제 2 와 같음). 상수 G_FORCE = 9.80665 가 Define.h 에 있다.

27

발표 대본

부록 · 약 50초

제공된 좌표변환 코드에서 이번에 쓰는 함수와, 실제로 컴파일해서 확인한 주의점입니다.

쓰는 건 다섯 개입니다. 위경도와 ECEF를 오가는 둘, 동체에서 NED, NED에서 ECEF, 그리고 결과 그림용으로 ECEF에서 NED. 안테나 계열과 ENU 계열, 플랫폼 보상 함수는 이번엔 안 씁니다.

함정이 넷입니다. 위경도에서 ECEF로 가는 함수는 경도가 첫째 인자입니다. 구조체는 위도가 먼저라 바꿔 넣기 쉬운데, 바꾸면 북위 54도 동경 212도라는 엉뚱한 점이 나옵니다. 자세각 인자는 롤, 요, 피치 순서입니다. 모든 각도는 라디안입니다. 그리고 NED에서 ECEF로 가는 함수는 플랫폼 위치를 더하므로 속도에는 0을 넣어야 합니다.

ECEF에서 위경도로 가는 함수는 반복을 5번 고정으로 하는데, 우리 위치에서 오차가 0.03 밀리미터라 그대로 써도 됩니다.

부록 B. 용어 정리

발표에 나온 말들. 질문이 들어오면 이 표를 보면서 답한다

용어	뜻	이번 과제에서
하중배수 n (g)	안쪽으로 당기는 가속도가 중력가속도의 몇 배인가	기동의 Gravity Value. $a = n \cdot g$
각속도 ω	1초에 몇 도(라디안) 도는가	$\omega = n \cdot g / V$. 매 스텝 자세각 $\pm \omega \times 0.1 \text{ s}$
오일러 적분	새 값 = 지금 값 + 변화율 $\times$ 시간 간격	$p \leftarrow p + v \times \Delta t$. 스텝 안 속도 일정이라 정확
자세각 roll · pitch · yaw	동체 축이 수평면(NED)에 대해 돌아간 세 각	표적의 진행 방향. yaw 180° = 남쪽
ECEF	지구 중심 원점, 지구와 함께 도는 직교좌표 [m]	위치·속도를 갱신하는 자리
NED	한 지점의 북 · 동 · 아래 수평 좌표	표적 자신의 자리 기준. 매 스텝 다시 잡는다
타원체고	WGS-84 타원체 면에서 수직으로 잰 높이	과제의 "고도". 해발고도와 약 20 m 차이
회전행렬 R	한 좌표계의 축을 다른 좌표계 축으로 돌려 읽는 3×3	위치엔 $R \cdot p + t$, 속도엔 $R \cdot v$

이상입니다. 질문 환영합니다. 코드 · 문서 : radar-sw-study / Part3

28

발표 대본

부록 · 약 40초

오늘 나온 용어를 한 표로 모았습니다. 하중배수, 각속도, 오일러 적분, 자세각, ECEF, NED, 타원체고, 회전행렬. 발표 중에 설명한 뜻 그대로이고, 질문이 있으면 이 표를 보면서 답하겠습니다.

발표에서는 말하지 않는다. 질문이 들어올 때만 쓴다.

Q1. 왜 룬지쿠타 4차를 안 쓰고 오일러 한 줄로 하나?

룬지쿠타는 가속도가 위치의 함수일 때, 그러니까 미분방정식을 풀어야 할 때 필요하다. 위성 궤도가 그 경우다. 이번 과제는 속도를 매 스텝 우리가 직접 정해 주므로 스텝 안에서 속도가 상수이고, 상수 속도의 적분은 곱셈 한 번으로 정확하다. 남은 오차는 방향이 스텝 안에서 조금씩 변하는 선회 구간뿐인데, 0.1초에 0.84도 (3 g, 200 m/s) 라서 원호를 600개 직선으로 잇는 오차는 반경 대비 0.01 % 수준이다. 필요하면 스텝 중간 자세로 속도를 구하는 중점법 한 줄로 더 줄일 수 있다.

Q2. 롤 기동을 어떻게 해석했나?

롤은 속도 방향 축(동체 x) 둘레의 회전이라 궤적을 직접 바꾸지 않고 자세만 바꾼다. 대신 그 뒤에 오는 피치 기동의 방향을 바꾼다. 롤 90도 뒤 피치는 수평 선회가 된다. 실제 항공기 선회 방식(기울이고 당기기)과 같다. 롤 각속도도 다른 축과 같이 $n \cdot g / V$ 로 두었는데, 이는 정의의 문제라 다른 값(예: 고정 롤 속도)으로 바뀌도 구조는 같다.

Q3. g 값의 부호, 기동이 겹칠 때, 끝난 뒤는?

부호는 방향이다. 양수면 오른쪽(요), 기수 들기(피치), 오른쪽 날개 내리기(롤), 음수면 반대. 겹치면 배열에서 먼저 적힌 기동 하나만 적용한다. 끝나면 그 순간 자세를 유지하고 직진한다. 원래 방향으로 복귀시키려면 반대 부호 기동을 하나 더 적으면 된다. 셋 다 과제가 정해 주지 않아서 우리가 정한 규칙이고, 코드 주석과 발표에 명시한다.

Q4. 11 m 떠오른다는 값의 근거는?

접선을 따라 거리 d 만큼 가면 지표면은 $d^2/(2R)$ 만큼 아래로 내려간다. $d = 12,000 \text{ m}$, $R = 6,371,000 \text{ m}$ 이면 11.3 m. 참고 코드로 실제로 60초 적분한 값이 $311.33 \text{ m} - 300 \text{ m} = 11.33 \text{ m}$ 로 일치한다. 매 스텝 위경도를 NED를 다시 잡으면 300.02 m 다. 남은 2 cm 는 스텝마다 접선 방향으로 20 m 를 가서 생기는 $20^2/(2 \times R) \times 600 = 1.9 \text{ cm}$ 이고, 이론값과 같다.